MiniTorch PagedAttention 项目中文指南（单文件版）

这份文档是英文多文件指南的中文整合版，面向两类读者：一类是不熟悉 PagedAttention 的读者，另一类是只大概了解 Transformer、但希望真正看懂本项目每个实现环节的读者。

项目路径是 `11868-course-project/` 。英文分篇入口在 README.md，根目录总览在 ../README.md。本文件把主要内容合并到一个中文文档里，阅读时不需要频繁跳转。

阅读路线

如果你是第一次读这个项目，建议按下面顺序：

1. 先理解 MiniTorch 在这里扮演什么角色。
2. 再理解 Decoder-only Transformer、prefill、decode 和 KV cache。
3. 然后读 `BlockManager` ，因为它是整个 PagedAttention 实现的中心。
4. 接着读 Python 参考实现和 CUDA 实现。
5. 最后看模型集成、前缀缓存、内存/容量分析、运行命令和 API。

最重要的一句话是：PagedAttention 不改变注意力的数学公式，它改变的是 KV cache 的内存管理方式。序列在逻辑上仍然连续，但物理 KV 存储可以分散在固定大小的块里；block table 负责把逻辑 token 位置翻译成物理 block 和 slot。

项目实现了什么

这个仓库在 MiniTorch 教学框架里实现了 PagedAttention 的核心机制，包括：

- 固定大小物理 KV block 的分配器。
- 每个序列自己的 block table。
- Python 参考版 PagedAttention。
- 可选的 CUDA decode kernel 和 stateful runtime。
- 集成 PagedAttention 的 decoder-only language model。
- 前缀缓存、block 共享、fragmentation 和 memory accounting。
- inference、benchmark、plot 和测试脚本。

核心文件：

路径	作用
../minitorch/block_manager.py	KV block 分配、释放、前缀缓存、fragmentation 和内存统计。

路径	作用
../minitorch/paged_attention.py	标准注意力、Python paged attention、CUDA ctypes wrapper、多头注意力模块。
../minitorch/transformer.py	decoder-only LM，包含 prefill、decode、prefix-hit grouping 和 generate。
../src/paged_attention.cu	自定义 CUDA PagedAttention kernel 和 runtime API。
../project/run_inference.py	合成 token 的 inference/correctness 驱动。
../project/run_benchmark.py	通用 throughput、latency、fragmentation、capacity、prefix-cache benchmark。
../project/run_rigorous_benchmark.py	用于报告/海报图的六组更严谨实验。

MiniTorch 基础

本项目不是直接用 PyTorch 写模型，而是用 MiniTorch。MiniTorch 是一个教学版深度学习框架，它提供类似 PyTorch 的 `Tensor` 、 `Module` 、 `Parameter` 、backend 和 tensor operation。

为什么 MiniTorch 很重要

项目里有两种主要数组：

1. MiniTorch `Tensor` ：用于模型计算，例如 embedding、linear、attention、LayerNorm。
2. NumPy array：用于 `BlockManager` 的 KV cache，以及作为 Python 和 ctypes/CUDA 之间的桥。

典型流程：

```
idx_np = np.array([[1, 2, 3]], dtype=np.float32)
idx = minitorch.tensor_from_numpy(idx_np, backend=backend)
logits = model.forward_prefill(idx, block_manager, seq_ids=[0])
logits_np = logits.to_numpy()
```

Tensor

`minitorch/tensor.py` 定义用户直接接触的 `Tensor` 。每个 Tensor 关心这些信息：

- shape，例如 `(batch, seq_len, n_embd)` 。
- backend，例如 `TensorBackend(FastOps)` 或 `TensorBackend(CudaKernelOps)` 。
- `view` 、 `permute` 、 `contiguous` 、 `sum` 、 `to_numpy` 等方法。
- 自动求导历史；虽然本项目主要跑 inference，但 Tensor 仍然带着 MiniTorch 框架元数据。

注意力代码里经常 reshaping 和 permuting。例如 `PagedMultiHeadAttention.forward_prefill` 会把 hidden state 从 `(batch, seq_len, n_embd)` 投影成 Q/K/V，再整理成 `(batch, n_head, seq_len, head_dim)` 做 attention。

TensorData

`minitorch/tensor_data.py` 负责更底层的 storage、shape、stride 和 indexing。多数项目代码不会直接操作 `TensorData`，但只要你看到 `view`、`permute` 或 `contiguous`，背后就在依赖这些低层规则。

一个常见细节是：`permute` 之后 Tensor 可能不是 contiguous 的。如果后面要 `view` 成新形状，代码通常会先调用 `.contiguous()`。

Backend

MiniTorch 用 `TensorBackend` 把“张量语义”和“执行实现”分开。

Backend	创建方式	用途
CPU fast path	`minitorch.TensorBackend(minitorch.FastOps)`	大多数测试和 CPU 示例默认用它。
CUDA tensor backend	`minitorch.TensorBackend(minitorch.CudaKernelOps)`	使用编译出的 MiniTorch CUDA primitive。

不要混淆两个 backend 概念：

- `--backend cpu|cuda` 控制 MiniTorch tensor operation 在 CPU 还是 CUDA tensor backend 上跑。
- `--decode-backend ref|cuda` 控制 PagedAttention decode 用 Python 参考实现还是自定义 CUDA kernel。

也就是说，MiniTorch tensor backend 和 PagedAttention decode backend 是两条不同的开关。

Module、Parameter 和基础层

MiniTorch 的 `Module` 类似 PyTorch 的 `nn.Module`。本项目中的这些类都是 Module：

- `PagedMultiHeadAttention`
- `FeedForward`
- `PagedTransformerLayer`
- `PagedDecoderLM`

模型使用的基础层来自 `minitorch/modules_basic.py`：

- `Linear`
- `Embedding`
- `Dropout`
- `LayerNorm1d`

这些层组成 Transformer。 `Linear` 用于 Q/K/V projection、attention output projection、feed-forward projection 和最终 `lm_head` 。 `Embedding` 用于 token embedding 和 position embedding。 `LayerNorm1d` 出现在 attention 和 feed-forward 之前。

Tensor 和 NumPy 的边界

本项目最需要留意的是 Tensor/NumPy 边界：

- `BlockManager.key_cache` 和 `BlockManager.value_cache` 是 NumPy array。
- `PagedMultiHeadAttention._write_kv_batch_to_cache` 会把 K/V Tensor 转成 NumPy，再写进 block cache。
- `paged_attention_ref` 会从 NumPy cache 按 block table gather K/V，然后包回 MiniTorch Tensor 调 `standard_attention` 。
- `PagedAttentionKernel` 会把输入整理成 contiguous NumPy array，再通过 ctypes 传给 CUDA shared library。

这种设计让项目更适合教学：内存管理用 NumPy 写得很直观，模型计算仍然留在 MiniTorch 里。

Transformer 和 KV Cache 基础

本项目实现的是 decoder-only language model。它接收 prompt，预测下一个 token，然后把新 token 接到上下文里继续预测。

Prefill 和 Decode

生成过程分两段：

1. **Prefill**：一次性处理整个 prompt。
2. **Decode**：每次只处理一个新 token。

例子：

```
Prefill input:  [t0, t1, t2, t3]
Sample:                         t4
Decode input:                   [t4]
Sample:                             t5
Decode input:                       [t5]
Sample:                                 t6
```

prefill 阶段可以并行处理 prompt 中所有位置；decode 阶段必须顺序进行，因为下一个 token 依赖上一个 token。

Attention 数学

每一层 Transformer 会把 hidden state 投影成：

- `Q` query：当前 token 想找什么。
- `K` key：每个历史 token 能被怎样匹配。

- `V` value：根据 attention weight 被复制的信息。

scaled dot-product attention：

```
scores  = Q * K^T / sqrt(head_dim)
weights = softmax(scores + mask)
output  = weights * V
```

本项目里 `standard_attention` 的输入 shape 是：

```
query: (batch, n_head, seq_q,  head_dim)
key:   (batch, n_head, seq_kv, head_dim)
value: (batch, n_head, seq_kv, head_dim)
```

prefill 时 `seq_q == seq_kv == prompt_len` 。decode 时 `seq_q == 1` ， `seq_kv == current_context_len` 。

Multi-Head Attention

模型维度满足：

```
n_embd = n_head * head_dim
```

每个 head 单独做 attention，输出再合并回 `n_embd` 。 `PagedMultiHeadAttention` 负责 Q/K/V projection、拆 head、做 attention、合并 head 和 output projection。

Causal Mask

decoder-only 模型不能让一个 token 看见未来 token。prefill 时位置 `i` 只能看 `0..i` ，不能看 `i+1..end` 。项目用一个 mask 表示：允许的位置加 `0` ，禁止的位置加 `-1e9` 。softmax 后，被禁止位置的概率几乎为 0。

decode 时，新 token 已经在序列末尾，所以它可以看见全部 cached tokens，包括自己刚写入的 K/V。

KV Cache

如果没有 KV cache，每次 decode 都要重新计算整个 prompt 和所有已生成 token 的 K/V，浪费很大。KV cache 的思路是：旧 token 的 K/V 一旦算出来，就保存下来。

```
After prefill:
  cache stores K/V for tokens 0..prompt_len-1

After decode step 1:
  append K/V for token prompt_len

After decode step 2:
  append K/V for token prompt_len+1
```

这样每个 decode step 只需要算新 token 的 Q/K/V，把新 K/V 追加到 cache，然后让 query attend 到全部 cache。

连续 KV Cache 的问题

简单的 serving 系统可以给每个 request 预留一大段连续 KV cache：

```
sequence 0: [slot 0, slot 1, ..., slot 1023]
sequence 1: [slot 0, slot 1, ..., slot 1023]
```

这很容易索引，但会浪费内存。如果一个序列实际只用了 80 tokens，却预留 1024 slots，大部分 KV 内存都空着。

每层每 token 的 KV 字节数近似为：

```
2 * n_head * head_dim * dtype_size
```

所有层合起来：

```
2 * n_layers * n_head * head_dim * dtype_size
```

连续 cache 的浪费来自它按最大长度预留，而不是按当前真实长度预留。

Paged KV Cache

PagedAttention 把每个序列的一大段连续 cache 替换成固定大小的物理 blocks。每个序列有一个 block table：

```
logical tokens:  0  1  2  3 | 4  5  6  7 | 8
logical blocks:        0    |      1      | 2
block table:     [physical block 5, physical block 2, physical block 9]
```

序列在逻辑上仍然是连续的，但物理存储可以分散。读取 token `token_index` 的 KV 时：

```
logical_block = token_index // block_size
slot_idx      = token_index % block_size
block_id      = block_table[logical_block]
```

然后读取：

```
key_cache[layer][block_id, slot_idx]
value_cache[layer][block_id, slot_idx]
```

这就是本项目的核心。

代码地图

PagedAttention 核心 Python 文件

文件	主要符号	角色
`minitorch/block_manager.py`	`KVBlock`，`BlockTable`，`PrefixCacheMatch`，`BlockManager`	管理物理 KV blocks、每个序列的 block table、prefix-cache metadata、allocation/free、fragmentation 和 memory accounting。
`minitorch/paged_attention.py`	`standard_attention`，`paged_attention_ref`，`PagedAttentionKernel`，`PagedMultiHeadAttention`	实现连续 attention、Python paged attention、CUDA wrapper、runtime 同步、prefix-aware prefill 和 decode attention。
`minitorch/transformer.py`	`FeedForward`，`PagedTransformerLayer`，`PagedDecoderLM`	把 paged attention 集成进 decoder-only LM。

MiniTorch 底座

这些文件来自教学框架，是项目运行的基础：

文件	作用
`minitorch/__init__.py`	导出 MiniTorch API 和项目模块，并尽量先初始化 PyTorch CUDA，避免 CUDA runtime/driver 初始化顺序问题。
`minitorch/tensor.py`	用户层 Tensor、NumPy 转换、operators、view、permute 和 backend dispatch。
`minitorch/tensor_data.py`	storage、shape、stride、indexing 和 broadcasting。
`minitorch/tensor_ops.py`	Backend 抽象以及 map/zip/reduce primitive。
`minitorch/fast_ops.py`	CPU fast tensor operations。

文件	作用
`minitorch/cuda_kernel_ops.py`	CUDA kernel-backed tensor operations 和 lazy PyCUDA loading。
`minitorch/modules_basic.py`	`Linear` 、 `Embedding` 、 `Dropout` 、 `LayerNorm1d` 。
`minitorch/modules_transfomer.py`	非 paged 的 transformer baseline，文件名里 `transfomer` 是原仓库拼写。
`minitorch/nn.py`	`softmax` 、 `dropout` 、 `GELU` 等神经网络函数。

CUDA 和构建

文件	作用
`src/paged_attention.cu`	项目自定义 PagedAttention CUDA kernel 和 stateful runtime。
`src/combine.cu`	MiniTorch CUDA tensor primitive。
`src/softmax_kernel.cu`	inherited attention softmax kernels。
`src/layernorm_kernel.cu`	inherited LayerNorm kernels。
`compile_cuda.sh`	编译 `combine.so` 、 `softmax_kernel.so` 、 `layernorm_kernel.so` 、 `paged_attention.so` 。

Project scripts

文件	作用
`project/run_inference.py`	用随机 token 运行 prefill/decode/generation，并支持 correctness check。
`project/run_benchmark.py`	通用 benchmark：throughput、latency、fragmentation、max batch、baseline、prefix cache。
`project/run_rigorous_benchmark.py`	六组 report/poster 实验。
`project/run_gpu_kv_benchmark.py`	直接调用 CUDA runtime，测量 GPU-resident KV cache 的 decode attention microbenchmark。
`project/contiguous_kv_baseline.py`	HuggingFace 风格的 contiguous KV cache baseline。
`project/plot.py`	绘制通用 benchmark CSV。
`project/plot_rigorous_figures.py`	绘制 report/poster figures。
`project/plot_gpu_kv_results.py`	绘制 GPU-only KV cache microbenchmark 图。

测试

文件	覆盖内容
`tests/test_block_manager.py`	allocation、free、block table、K/V 写入、fragmentation、prefix cache、eviction。
`tests/test_paged_attention.py`	standard attention、paged reference、module prefill/decode、transformer integration、CUDA parity。
`tests/test_parity.py`	NumPy MHA parity、decode/full-recompute parity、KV memory accounting。
`tests/test_benchmark.py`	allocator speed 和 free/reallocate performance。

BlockManager 详解

`BlockManager` 是项目的核心。它把 KV cache 从“每个序列一大段连续内存”变成“全局固定大小物理 block 池 + 每个序列自己的 block table”。

它解决的问题

naive cache 会按最大序列长度为每个 active sequence 预留内存。实际 prompt 往往短很多，或者长度不均匀，于是大量 slot 被浪费。BlockManager 只在 token 真的存在时分配 block。

```
sequence tokens:      0  1  2  3 | 4  5
logical blocks:             0    |   1
block table:          [physical block 7, physical block 2]
physical cache pool:  block 0, block 1, block 2, ..., block 7, ...
```

模型看到的是连续 token；分配器看到的是物理 blocks。

`KVBlock`

`KVBlock` 只是一个物理 block 的 metadata，不直接保存 K/V 向量。真正的 K/V 在 `BlockManager.key_cache` 和 `BlockManager.value_cache` 。

关键字段：

- `block_id` ： 物理 block 编号。
- `block_size` ： 每个 block 的 token slot 数。
- `ref_count` ： 有多少 active/cached owner 引用它。
- `num_filled` ： 当前有效 token slot 数。
- `n_head` 、 `head_dim` ： K/V shape metadata。
- `block_hash` 、 `is_prefix_cached` ： 前缀缓存 metadata。

常用属性：

- `is_full`： `num_filled >= block_size` 。
- `num_empty_slots`： 剩余 slot 数。

`BlockTable`

`BlockTable` 是一个序列自己的 logical-to-physical 映射。

```
BlockTable(seq_id=10, block_ids=[7, 2, 11])
```

表示：

```
sequence 10 logical block 0 -> physical block 7
sequence 10 logical block 1 -> physical block 2
sequence 10 logical block 2 -> physical block 11
```

`PrefixCacheMatch`

`lookup_prefix_blocks` 返回 `PrefixCacheMatch`：

- `block_ids`： 可以复用的物理 blocks。
- `cached_token_count`： 这些 blocks 覆盖的 token 数。

由于只缓存 full block， `cached_token_count` 总是 `len(block_ids) * block_size` 。

`BlockManager` 的主要状态

```
blocks:             block_id -> KVBlock metadata
free_block_ids:     普通空闲 physical block id 列表
block_tables:       seq_id -> BlockTable
context_lens:       seq_id -> 当前序列 token 数
key_cache[layer]:   (num_blocks, block_size, n_head, head_dim)
value_cache[layer]: (num_blocks, block_size, n_head, head_dim)
```

每一层 Transformer 都有自己的 K cache 和 V cache，所以 `BlockManager` 需要 `num_layers` 参数。

Cache shape

每层：

```
key_cache[layer].shape   == (num_blocks, block_size, n_head, head_dim)
value_cache[layer].shape == (num_blocks, block_size, n_head, head_dim)
```

读取一个 token 的 key：

```
key_cache[layer][block_id, slot_idx, :, :]
```

结果 shape 是 `(n_head, head_dim)` 。

分配流程

`allocate_block` 负责拿一个物理 block：

1. 如果普通 free list 空了，尝试 `evict_cached_block_if_needed` 。
2. 仍然没有 free block 就抛 `RuntimeError` 。
3. 弹出一个 free block id。
4. 重置 metadata： `ref_count = 1` ， `num_filled = 0` 。
5. 把该 block 在每一层的 K/V cache slice 清零。
6. 返回 `KVBlock` metadata。

`allocate_blocks_for_sequence(seq_id, num_tokens)` 给初始 prompt 分配足够 blocks：

```
num_blocks = ceil(num_tokens / block_size)
```

例如 `num_tokens=10` 、 `block_size=4` ，会分配 3 个 blocks， `num_filled` 分别是 `4` 、 `4` 、 `2` 。如果中途 OOM，它会回滚已经分配的 blocks，避免 allocator 状态半坏。

`append_token_to_sequence(seq_id)` 用于 decode 前给一个新 token 预留 slot：

1. 如果序列还没有 block，就分配一个。
2. 否则检查最后一个 block。
3. 如果最后一个 block 满了，就分配新 block 并 append 到 block table。
4. 增加该 block 的 `num_filled` 。
5. 增加 `context_lens[seq_id]` 。

`free_sequence(seq_id)` 释放序列：

1. 遍历 block table，减少每个 block 的 `ref_count` 。
2. 如果 block 引用为 0 且是 prefix-cached，把它放入 `cached_free_lru` ，保留可复用 K/V。
3. 如果引用为 0 且不是 cached，清零 K/V 并放回 `free_block_ids` 。
4. 删除序列的 block table、context len 和 prefix-cache info。

逻辑位置到物理位置

`get_physical_location(seq_id, token_index)` 是最关键的翻译：

```
logical_block = token_index // block_size
slot_idx = token_index % block_size
block_id = block_tables[seq_id].block_ids[logical_block]
return block_id, slot_idx
```

所有 K/V cache 的读写最终都依赖这个映射。

写入 K/V

`write_kv_slot(layer_id, block_id, slot_idx, key, value)` 把一个 token 的 K/V 写到已知物理位置：

```
key_cache[layer_id][block_id, slot_idx, :, :] = key
value_cache[layer_id][block_id, slot_idx, :, :] = value
```

`write_token_kv(seq_id, layer_id, key, value)` 则把 append 和 write 合在一起：先为序列追加一个 slot，再写入对应 layer 的 K/V。

CUDA metadata

CUDA kernel 需要 rectangular 的 block table array。 `get_block_table_array(seq_ids, pad_value=-1)` 会构造：

```
seq 0 block ids: [3, 4]
seq 1 block ids: [7]

array:
[[ 3,  4],
 [ 7, -1]]
```

kernel 会根据每个序列的 `context_lens` 只读取真实 token 覆盖的 entries，padding 不应被访问。

Fragmentation 和 memory metrics

internal fragmentation：

```
(allocated_token_slots - used_token_slots) / allocated_token_slots
```

例如 `block_size=16` 、 `seq_len=33` ，需要 3 blocks，分配 48 slots，使用 33 slots，内部碎片是 `15 / 48` 。

`compute_kv_memory(max_seq_len)` 比较 paged cache 和 naive contiguous baseline：

```
paged_bytes = num_used_blocks * block_size * 2 * n_head * head_dim * dtype_size * num_layers
contiguous_bytes = num_active_sequences * max_seq_len * 2 * n_head * head_dim * dtype_size * num_layers
memory_savings_ratio = 1 - paged_bytes / contiguous_bytes
```

BlockManager 不变量

- 每个 active `seq_id` 有一个 `BlockTable` 和一个 `context_lens` entry。
- `context_lens[seq_id]` 是该序列当前逻辑 token 数。
- 每个物理 block 的 `num_filled` 记录该 block 内有效 slots。
- `key_cache` 和 `value_cache` 的索引顺序是 layer、physical block、slot、head、head_dim。
- prefix-cache block 可以在原序列释放后继续存在。
- full-block prefix reuse 是安全的，因为复用的 block 在逻辑上是只读前缀。

Python PagedAttention 详解

`minitorch/paged_attention.py` 同时包含三层概念：

1. `standard_attention`： 连续 K/V 上的 scaled dot-product attention。
2. `paged_attention_ref`： 慢但清晰的 Python 参考版，从 block table gather K/V。
3. `PagedMultiHeadAttention`： Transformer 实际使用的多头注意力模块。

`standard_attention`

输入 shape：

```
query: (batch, n_head, seq_q,  head_dim)
key:   (batch, n_head, seq_kv, head_dim)
value: (batch, n_head, seq_kv, head_dim)
mask:  broadcastable to (batch, n_head, seq_q, seq_kv)
```

计算：

```
scores  = sum(query * key) / sqrt(head_dim)
scores += mask, if provided
weights = softmax(scores, dim=3)
output  = sum(weights * value)
```

它用于 prefill，也用于 correctness reference。

`paged_attention_ref`

`paged_attention_ref` 是理解 PagedAttention 最直接的函数。它不追求速度，而是把算法写清楚。

核心逻辑：

```
for token_idx in range(context_len):
    logical_block_idx = token_idx // block_size
    slot_idx = token_idx % block_size
    block_id = block_table[logical_block_idx]
    gathered_keys.append(key_cache[layer_id][block_id, slot_idx])
    gathered_values.append(value_cache[layer_id][block_id, slot_idx])
```

gather 完以后，它把 K/V 包回 MiniTorch Tensor，然后调用 `standard_attention` 。CUDA kernel tests 和 `--compare-to-ref` 都以这个函数作为正确性锚点。

`PagedMultiHeadAttention`

这个模块是 Transformer 直接调用的 attention layer。关键参数：

- `n_embd` ： 模型维度。
- `n_head` ： head 数。
- `head_dim = n_embd // n_head` 。
- `block_size` ： 每个 KV block 的 token 数。
- `layer_id` ： 访问 `BlockManager.key_cache[layer_id]` 和 `value_cache[layer_id]` 。
- `decode_backend` ： `"ref"` 或 `"cuda"` 。
- `compare_to_ref` ： CUDA decode 时是否同时跑 Python reference 并比较误差。

Prefill path

`forward_prefill(x, block_manager, seq_ids)` 处理完整 prompt。

输入：

```
x: (batch, seq_len, n_embd)
```

流程：

1. flatten 到 `(batch * seq_len, n_embd)` 。
2. 用 `q_proj` 、 `k_proj` 、 `v_proj` 计算 Q/K/V。
3. reshape 成 heads，再 permute 到 `(batch, n_head, seq_len, head_dim)` 。
4. 构造 causal mask。
5. 调 `standard_attention(q, k, v, mask)` 。
6. merge heads，调 `out_proj` 。
7. 把 K/V 转成 NumPy，顺序为 `(batch, seq_len, n_head, head_dim)` 。
8. 用 `_write_kv_batch_to_cache` 写入 block manager。

prefill 的 attention 计算本身还是连续 attention；paging 的作用是把算出的 K/V 存进 blocks，供 decode 读取。

Decode path

`forward_decode(x, block_manager, seq_ids)` 每次处理一个新 token。

输入：

```
x: (batch, 1, n_embd)
```

重要顺序： `PagedDecoderLM.forward_decode` 在 layer 运行前已经调用 `append_token_to_sequence` 预留了当前 token 的 slot。之后每一层 attention 把本层 K/V 写到同一个逻辑 token 位置。

流程：

1. 为一个 token 投影 Q/K/V。
2. 把新 K/V 转成 `(batch, n_head, head_dim)` 的 NumPy arrays。
3. 对每个序列取 final token position： `context_len - 1` 。
4. 调 `get_physical_location` 找到 `(block_id, slot_idx)` 。
5. 把新 K/V 写入该 layer 的 cache。
6. 如果 CUDA runtime 模式启用，更新 device-side slot。
7. 调 `_decode_attention_ref` 或 `_decode_attention_kernel` 。
8. 如果 `compare_to_ref=True` ，比较 CUDA output 和 Python reference。
9. merge heads，调 `out_proj` 。

输出 shape 是 `(batch, 1, n_embd)` 。

Prefix-aware prefill

`forward_prefill_with_prefix_batch` 处理“前缀已经在 cache 中，只需计算 suffix/work tokens”的情况。

Python reference path：

1. 计算 suffix tokens 的 Q/K/V。
2. 用 `_gather_cached_prefix_kv_batch` 从 block manager gather cached prefix K/V。
3. 拼接 prefix K/V 和 suffix K/V。
4. 构造 causal mask：suffix token `i` 可见 `prefix_token_count + i + 1` 个 tokens。
5. 用 `standard_attention` 对完整上下文做 attention。

CUDA path 会调用 prefix-aware prefill kernel，让 kernel 从 runtime cache 读 prefix blocks，并从临时 suffix arrays 读新 K/V。

CUDA Kernel 和 Runtime

CUDA path 有两半：

1. Python 侧 `PagedAttentionKernel` ：加载 shared library，注册 ctypes 签名，提供高层方法。

2. C++/CUDA 侧 `src/paged_attention.cu`：实现 GPU kernels 和 `PagedAttentionRuntime`。

构建输出

`compile_cuda.sh` 会生成：

```
minitorch/cuda_kernels/combine.so
minitorch/cuda_kernels/softmax_kernel.so
minitorch/cuda_kernels/layernorm_kernel.so
minitorch/cuda_kernels/paged_attention.so
```

PagedAttention library 使用：

```
nvcc -std=c++17 -o minitorch/cuda_kernels/paged_attention.so \
  --shared src/paged_attention.cu -Xcompiler -fPIC --cudart shared
```

`--cudart shared` 很重要，因为 Python 进程里可能已经有 PyTorch 的 CUDA runtime state，也可能有 numba 的 driver API state。共享 CUDA runtime 可以避免一份静态 runtime 看不到已初始化设备状态的问题。

CUDA context import order

`minitorch/__init__.py` 会尽量先初始化 PyTorch CUDA，再导入可能触碰 `numba.cuda` 的模块。这样可以避免 runtime API 和 driver API 初始化顺序造成的错误，例如明明有 GPU，却在 `cudaSetDevice` 或 `cudaMalloc` 时报告没有 CUDA-capable device。

如果 CUDA 行为很奇怪，优先检查：

- `minitorch/cuda_kernels/paged_attention.so` 是否存在。
- `compile_cuda.sh` 是否用 `--cudart shared` 编译 `paged_attention.cu`。
- 当前进程 import 的是不是本仓库的 `minitorch`。
- 环境是否能看到 CUDA device。

Stateless forward

stateless 模式每次调用都传入 host-side cache 和 metadata：

```
kernel.forward(
    query,
    key_cache,
    value_cache,
    block_tables,
    context_lens,
    block_size,
    max_context_len,
)
```

这种方式简单，适合测试 ABI 和正确性，但每次都要复制大量 cache 数据。

Stateful runtime

stateful 模式在 C++ 侧创建 `PagedAttentionRuntime` ，持有 device buffers：

```
d_key_cache
d_value_cache
d_block_tables
d_context_lens
d_query
d_output
d_prefill_query
d_prefill_suffix_key
d_prefill_suffix_value
d_prefill_output
```

Python 侧只更新变化的内容：

- `upload_block` 上传一个完整 physical block。
- `update_slot` 更新一个 decode slot。
- `update_metadata` 上传 block tables 和 context lengths。
- `forward` 用 device-resident cache 运行 decode attention。

`PagedMultiHeadAttention` 用 `_runtime_valid_block_ids` 记录哪些 blocks 在 device 端已经有效，避免每步都上传全 cache。runtime 重建或切换 block manager 时，这个集合会被清空。

Decode kernel

`paged_attention_v1_kernel` 为 decode query 计算 attention。

launch 形状：

```
grid  = (batch_size, n_head)
block = max(WARP_SIZE, min(head_dim, 1024)) threads
```

每个 CUDA thread block 处理一个 `(sequence, head)` pair。

kernel 对每个 context token 做和 Python reference 一样的翻译：

```
int logical_block = token_idx / block_size;
int slot_in_block = token_idx % block_size;
int physical_block = seq_block_table[logical_block];
```

然后用 pointer arithmetic 找到 K/V：

```
cache + physical_block * (block_size * n_head * head_dim)
      + slot_in_block * (n_head * head_dim)
      + head_idx * head_dim
```

三遍 attention

decode kernel 使用 dynamic shared memory，逻辑上分成：

```
logits[max_context_len]
out_accum[head_dim]
q_shared[head_dim]
warp_scratch[num_warps]
```

算法三步：

1. score pass：对每个 context token 计算 `Q dot K` 并保存 logits。
2. softmax pass：减 global max、取 exp、reduce sum、归一化。
3. value pass：累加 `sum(weight[token] * V[token])` 。

`warp_reduce_max` 和 `warp_reduce_sum` 用 `__shfl_xor_sync` 做 warp 内归约，warp leaders 再把结果写入 shared memory 做跨 warp 归约。

Prefix-prefill kernel

`paged_attention_prefill_with_prefix_kernel` 处理“prefix 已 cached，suffix 需要计算”的 batched rows。

launch 形状：

```
grid = (batch_size * work_len, n_head)
```

每一 row 对应一个 batch item 的一个 suffix token。它的可见上下文是：

```
cached prefix tokens + suffix tokens up to this local row
```

如果 `token_idx < prefix_token_count` ，kernel 从 paged runtime cache 读 K/V；否则从本次调用传入的 suffix K/V arrays 读。

Transformer 集成

`minitorch/transformer.py` 把 block manager 和 paged attention 集成成 decoder-only LM。

模型结构：

```
token ids
  -> token embedding + position embedding
  -> PagedTransformerLayer 0
  -> PagedTransformerLayer 1
  -> ...
  -> final LayerNorm
  -> lm_head
  -> logits
```

每个 layer 使用 pre-layer normalization：

```
x -> LayerNorm -> Attention -> residual add
  -> LayerNorm -> FeedForward -> residual add
```

`FeedForward`

MLP 子层：

```
Linear(n_embd, 4 * n_embd)
GELU
Linear(4 * n_embd, n_embd)
Dropout
```

它先把 `(batch, seq_len, n_embd)` reshape 成 `(batch * seq_len, n_embd)` 做 linear，再 reshape 回来。

`PagedTransformerLayer`

包含：

- `ln_1`
- `ln_2`
- `attention: PagedMultiHeadAttention`
- `ff: FeedForward`

prefill 和 decode 分别有不同 forward，因为 attention 行为不同。

`forward_prefill`：

1. `ln_1` 。
2. `attention.forward_prefill` ，使用 standard attention 并写 prompt K/V。
3. residual add。
4. `ln_2` 。
5. feed-forward。
6. residual add。

`forward_decode` 结构相同，但 attention 会通过 block table 读取历史 K/V。

`PagedDecoderLM`

完整模型包含：

- `token_embeddings`
- `position_embeddings`
- `layers`
- final `ln`
- `lm_head`

注意：模型不拥有 block manager。 `BlockManager` 由 caller 创建并传入 prefill/decode，这样 caller 可以控制 KV 容量和序列生命周期。

`_embed`

`_embed(idx, start_pos=0)` 做 token embedding + positional embedding。

prefill：

```
idx shape: (batch, prompt_len)
position ids: 0..prompt_len-1
```

decode：

```
idx shape: (batch, 1)
position id: current token position
```

`start_pos` 用来保证 decode token 的位置 embedding 与完整序列位置一致。

Prefill without prefix hit

`forward_prefill(idx, block_manager, seq_ids)` 先检查 prefix cache。如果所有序列都 miss，就走简单路径：

1. 为每个序列调用 `allocate_blocks_for_sequence` 。
2. embed 整个 prompt。
3. 逐层调用 `PagedTransformerLayer.forward_prefill` 。
4. 所有层写完 K/V 后，调用 `publish_sequence_prefix_blocks` 发布 full prompt blocks。
5. final LayerNorm + `lm_head` 。
6. 返回 `(batch, seq_len, n_vocab)` logits。

Prefill with prefix hit

如果某些序列命中 prefix cache， `forward_prefill` 会把 batch 拆开：

- 没有命中的序列走普通 full prefill。
- 命中的序列按 `cached_token_count` 分组。

分组很重要，因为同一个 batched suffix computation 里 prefix 长度和 work length 必须一致。

对每个 hit group， `_forward_prefill_group_with_prefix`：

1. 设置 `work_start = cached_token_count` 。如果整段 prompt 都 cached，则保留最后一个 token 作为 work，以便产生 logits。
2. 切出 `idx_work = idx[:, work_start:]` 。
3. 用 `start_pos=work_start` embed work tokens。
4. 每层调用 `attention.forward_prefill_with_prefix_batch` 。
5. 对 work tokens 做 final norm 和 `lm_head` 。
6. 返回完整 shape 的 logits；被跳过的位置填 0，work 位置有真实 logits。

Decode path

`forward_decode(idx, block_manager, seq_ids, start_pos=0)` 要求每个序列只有一个新 token：

```
idx: (batch, 1)
```

流程：

1. 用 `start_pos` embed 新 token。
2. 如果某个 `seq_id` 还没有 active sequence，则创建空序列。
3. 对每个 `seq_id` 调 `append_token_to_sequence` 预留一个 slot。
4. 逐层调用 `PagedTransformerLayer.forward_decode` 。
5. final norm + `lm_head` 。
6. 返回 `(batch, 1, n_vocab)` logits。

slot 预留发生在所有层之前，因为每一层都要为同一个逻辑 token 位置写自己的 K/V。

Generation loop

`PagedDecoderLM.generate` 是一个静态 helper：

1. 设置 eval mode。
2. 把 prompt tensor 转成 Python list，便于累计输出。
3. 对完整 prompt 调 `forward_prefill` 。
4. 从最后一个 prompt position 的 logits sample 第一个新 token。
5. 重复 decode：把上一个 sampled token 包成 `(batch, 1)` ，调用 `forward_decode` ，sample 下一个 token。
6. 在 `finally` 中释放所有 active sequence，并关闭 CUDA decode runtime。

`finally` 很重要：即使 decode 中途报错，也会释放 blocks。

前缀缓存和共享

很多 serving workloads 会共享同一段开头，例如 system prompt、长 instruction template、retrieval preamble、多采样同一 prompt、beam search 的公共 trunk。前缀缓存让后续请求复用先前算好的 full-block KV。

共享单位

项目只共享完整 blocks。假设 `block_size=4` ，prompt 有 10 tokens：

```
tokens:        0 1 2 3 | 4 5 6 7 | 8 9
full blocks:   block A |  block B | partial tail
shared later:  yes     |  yes     | no
```

partial tail 不发布到 prefix cache。这样 mutation 规则简单：cached blocks 是只读 full blocks，decode append 会自然分配新 block。

Hash chain

`compute_block_hash_chain(token_ids, extra_hash=None)` 为每个 full logical block 算一个 hash。每个 hash 包含：

1. 上一个 block 的 hash。
2. 当前 block 的 token IDs。
3. 可选 `extra_hash` 。

```
block 0 hash = H(None, tokens[0:4], extra_hash)
block 1 hash = H(block0_hash, tokens[4:8], extra_hash)
block 2 hash = H(block1_hash, tokens[8:12], extra_hash)
```

这是 hash chain，而不是孤立地 hash 每个 token block。相同 token block 如果出现在不同前缀之后，会得到不同 hash。

Lookup、allocation、publish

`lookup_prefix_blocks(token_ids)` 从第一个 block 开始沿 hash chain 查找，直到第一次 miss，返回最长连续 cached prefix。

如果 block 0 和 1 命中、block 2 miss：

```
PrefixCacheMatch(block_ids=[id0, id1], cached_token_count=2 * block_size)
```

`allocate_sequence_with_prefix(seq_id, num_tokens, prefix_match)` 会创建一个 active sequence，它的前几个 logical blocks 指向已有 physical blocks，并对这些 blocks 增加 `ref_count` 。suffix 则分配新 blocks。

`publish_sequence_prefix_blocks(seq_id, token_ids)` 在 prefill 完成后发布 full blocks。必须等所有 layer 写完 K/V 后再发布，否则后来的请求可能复用到不完整的 cache。

Reference count 和 eviction

`ref_count` 是共享安全机制。block 的 owner 可能是 active sequence，也可能是 prefix cache metadata。

当一个序列释放时，如果某个 cached block 没有 active reference，但仍然可作为 prefix cache 复用，它会进入 `cached_free_lru` ，而不是直接回到普通 free list。

如果普通 free blocks 用完， `evict_cached_block_if_needed` 可以回收一个 cached-free block：

1. 从 LRU 队列弹出一个 cached block。
2. 删除 hash mappings。
3. 清除 `block_hash` 和 `is_prefix_cached` 。
4. 清零每一层的 K/V storage。
5. 放回普通 free list。

没有实现的内容

为了保持教学实现清晰，本项目没有实现：

- partial tail block sharing。
- 对 partially filled shared block 的任意 copy-on-write。
- prefix path 之外的 block deduplication。
- 生产级 scheduler 和复杂 eviction policy。

内存、容量和 Benchmark 解读

PagedAttention 的核心收益是内存管理。它不会改变 attention 数学，但会改变 KV cache 如何 reservation、use、share 和 reclaim。

每 token KV 字节数

一层一个 token：

```
bytes_per_token_per_layer = 2 * n_head * head_dim * dtype_size
```

所有层：

```
bytes_per_token = 2 * n_layers * n_head * head_dim * dtype_size
```

因为 `n_embd = n_head * head_dim` ，也可以写成：

```
bytes_per_token = 2 * n_layers * n_embd * dtype_size
```

Paged reservation

一个长度为 `L` 、block size 为 `B` 的序列需要：

```
ceil(L / B) blocks
```

预留 token slots：

```
ceil(L / B) * B
```

最后一个 block 没用完的 slots 就是 internal fragmentation。

例子： `block_size=16` ， `seq_len=33` ：

```
ceil(33 / 16) = 3 blocks
reserved slots = 48
used slots = 33
wasted tail slots = 15
```

Static contiguous reservation

简单 contiguous cache 通常按最大长度为每个 active sequence 预留：

```
reserved_slots = batch_size * max_seq_len
```

Paged allocation 按当前真实长度增长，内存浪费通常小很多。

Block budget 下的 capacity

固定 block 数下，最大 batch size 近似：

```
max_batch ~= num_blocks / ceil(seq_len / block_size)
```

例如 `num_blocks=512` ：

```
block_size=16, seq_len=128 -> 8 blocks per sequence  -> about 64 sequences
block_size=8,  seq_len=128 -> 16 blocks per sequence -> about 32 sequences
```

block size 越大，block table overhead 越小，整除时能容纳更多序列；但对于非对齐长度，尾部浪费也更大。

Prefix sharing 的容量收益

如果 N 个序列共享同一前缀：

```
without sharing: N sequences * prefix_blocks
with sharing:    1 shared prefix copy + N suffixes
```

parallel sampling 和 beam-search benchmark 模拟的就是这种 block-level sharing 的内存效果。

Benchmark families

`project/run_benchmark.py` 输出通用 CSV：

- throughput 和 latency。
- fragmentation sweep。
- max-batch search。
- correctness comparison。
- optional no-cache baseline。
- optional prefix-cache prefill comparison。

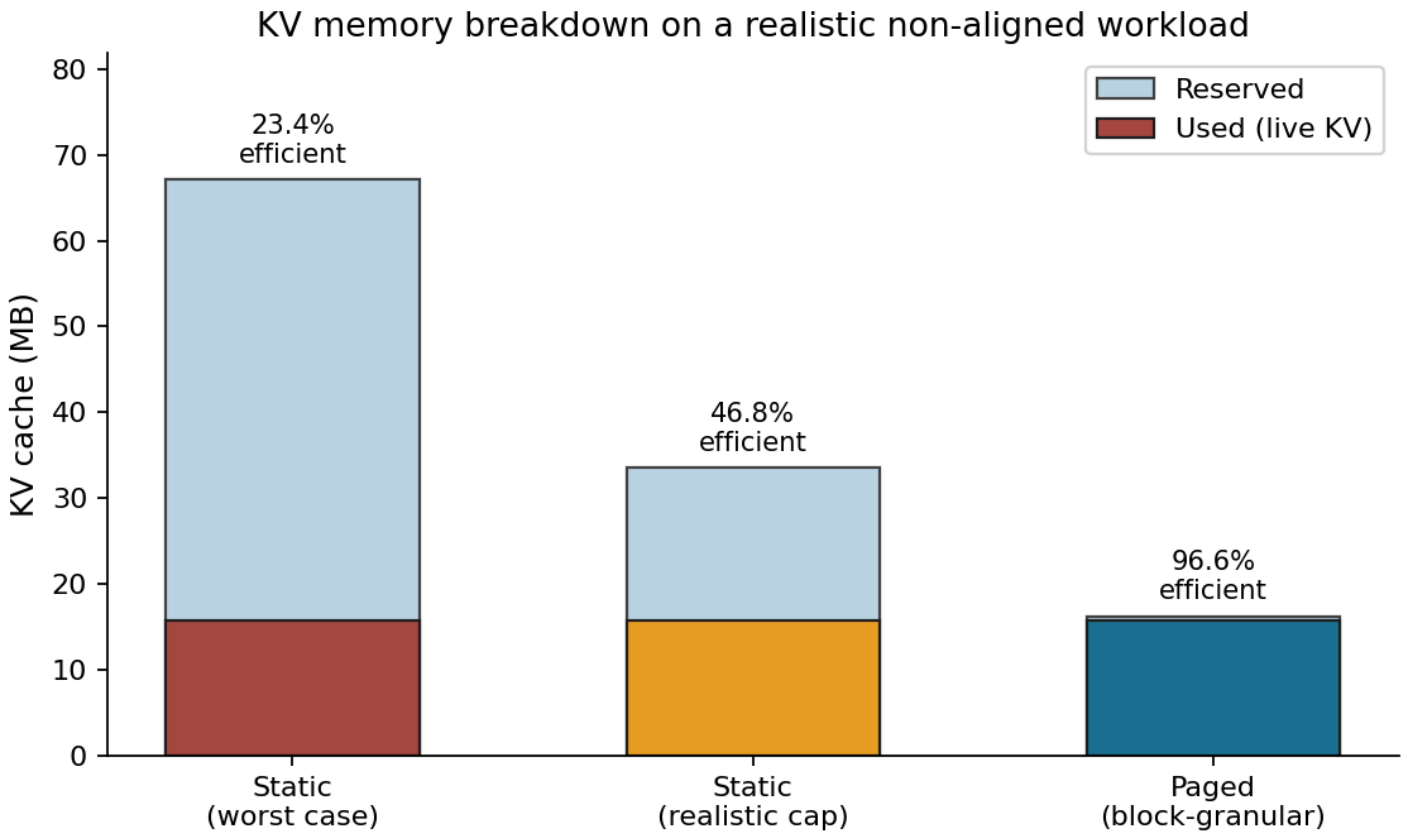
`project/run_rigorous_benchmark.py` 输出六组 report-grade experiments：

1. realistic non-aligned workload 的 memory breakdown。
2. 固定 KV block budget 下的 capacity curve。
3. paged decode 对比 no-cache 和 contiguous-KV baseline 的 decode speed。
4. 不同 shared ratio 下的 prefix prefill speedup。
5. parallel sampling memory。
6. beam search memory。

Rigorous benchmark 结果图解读

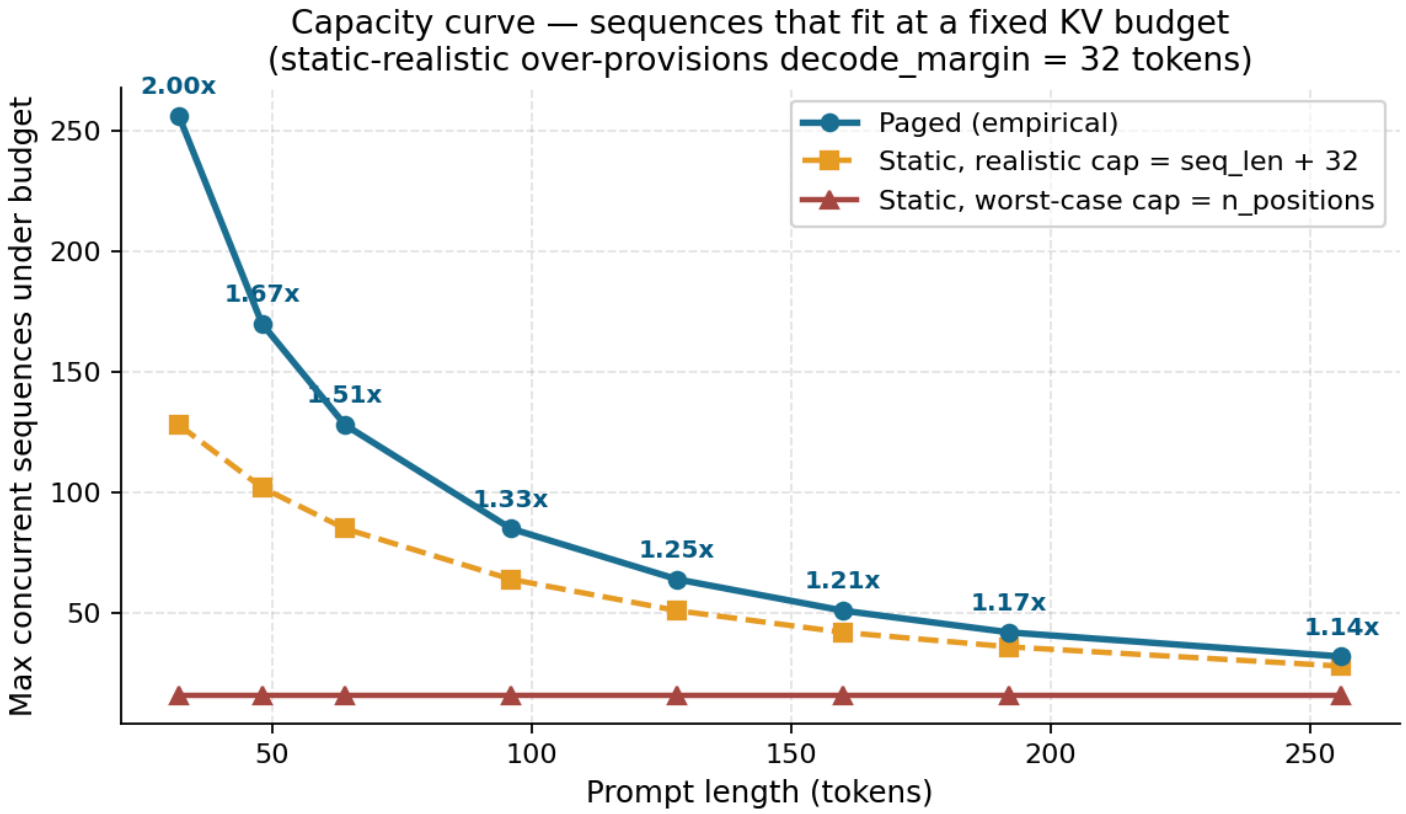
下面这些图由 `project/run_rigorous_benchmark.py` 生成 CSV，再由 `project/plot_rigorous_figures.py` 绘制到 `benchmarks/report_figures_v2/` 。它们主要回答一个问题：block-granular KV cache 在内存、容量、prefix reuse 和 branching 场景里到底带来什么收益。

Figure 1：KV memory breakdown



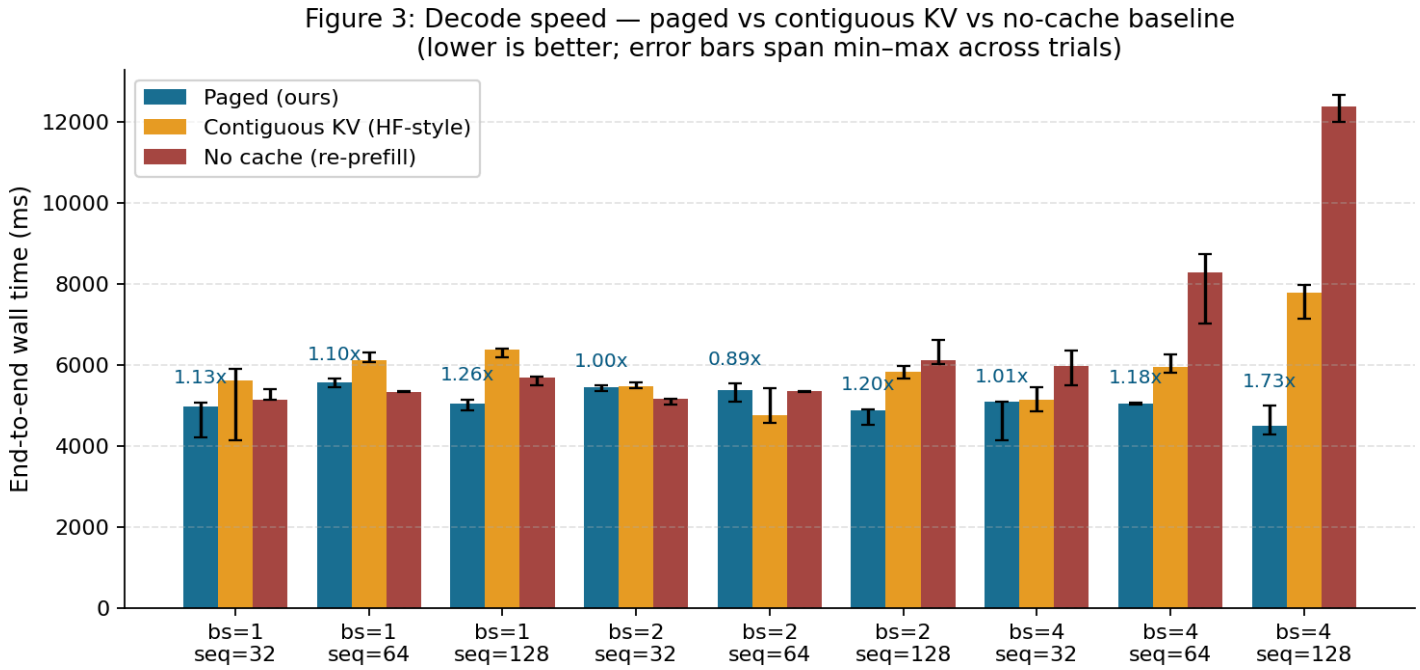
这张图是最核心的 allocator 结果。realistic non-aligned workload 的序列长度不是 block size 的整数倍，因此能同时暴露 static reservation 和 tail fragmentation 的浪费。Worst-case static reservation 只有 `23.4%` efficiency；realistic static cap 提升到 `46.8%`；paged block-granular allocation 达到 `96.6%`。这说明 PagedAttention 的主要收益首先不是“attention 算得更快”，而是 KV cache 不再按最大长度为每个 request 预留一整块 slab，只为真实存在的 tokens 分配 blocks。

Figure 2：固定 KV budget 下的 capacity curve



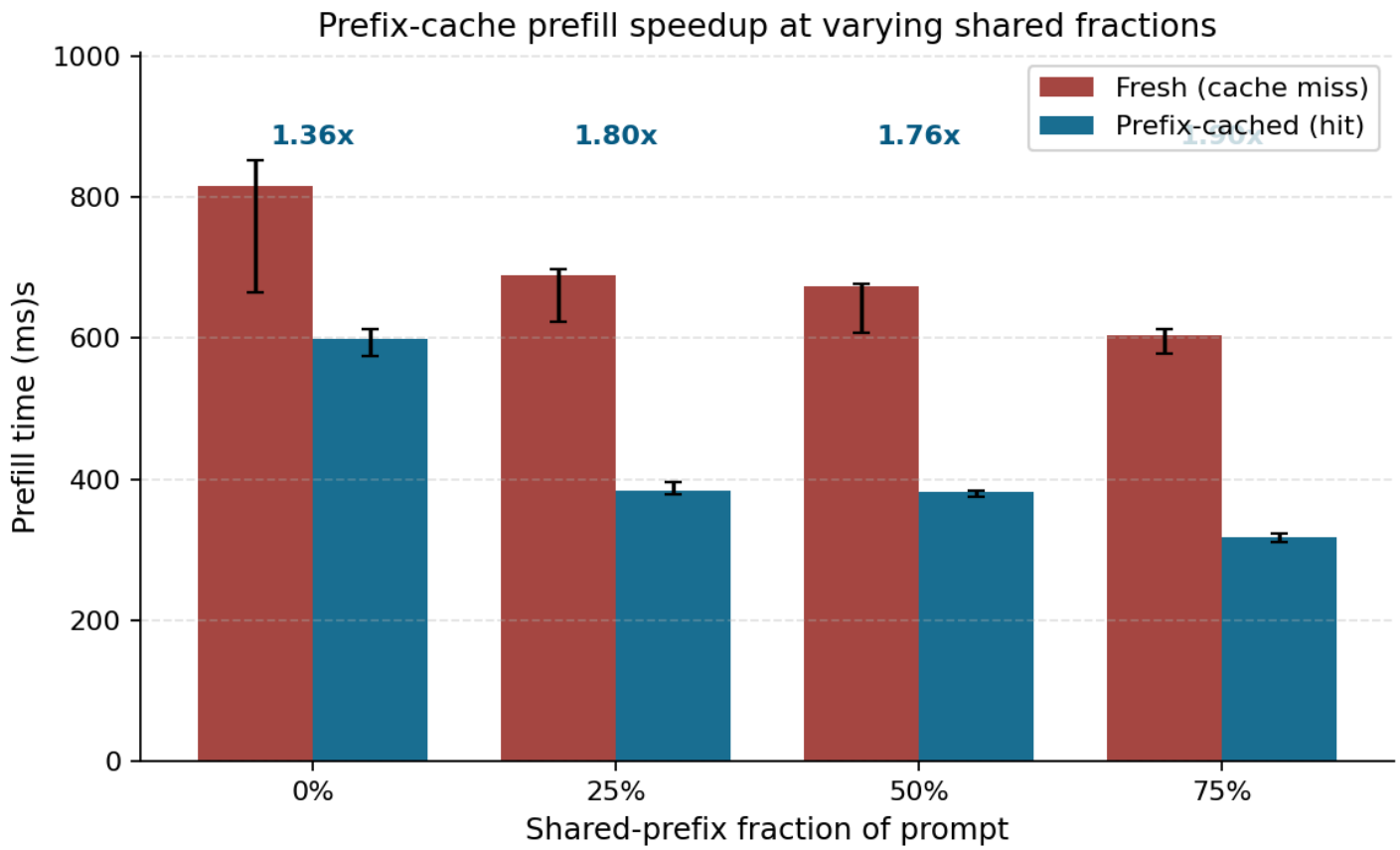
这张图把 KV memory budget 固定住，然后比较能同时容纳多少条序列。短 prompt 时 paged allocation 最有优势，因为 static baseline 的 `seq_len + 32` decode margin 相对真实长度很大，paged 方法最多能容纳 `2.00x` 的 concurrent sequences。随着 prompt length 增大，static over-provision 的相对比例变小，paged 的优势逐渐收敛，但在 `256` tokens 仍然有 `1.14x` 。这和 PagedAttention 论文的核心叙事一致：KV 管理越接近“按需分配”，serving admission headroom 越高。

Figure 3：Decode speed，对比 paged、contiguous KV 和 no-cache



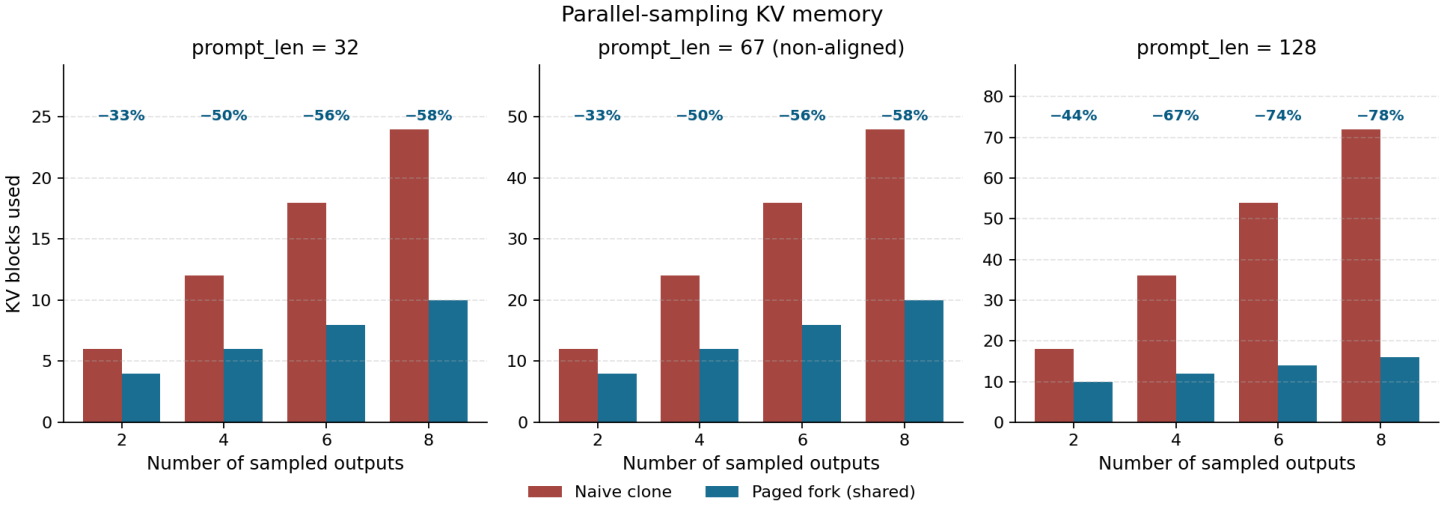
这张图要谨慎解读。Paged decode 通常比 no-cache re-prefill 更快，因为它复用了历史 K/V，而不是每一步重新处理完整上下文。和 contiguous KV baseline 相比，优势更小，有些配置甚至 contiguous 更快；这是合理的，因为本项目的 CUDA kernel 和 Python/MiniTorch runtime 是教学实现，没有 vLLM production kernel 的 layout、coalescing、online softmax、V2 partitioning 和 scheduler 优化。因此 Figure 3 更适合证明“KV cache 避免重复计算是有效的”，不适合用来声称本项目复现了 vLLM 的 production throughput。

Figure 4：Prefix-cache prefill speedup



这张图展示 shared prefix 命中后，只计算 suffix 的收益。shared-prefix fractions 从 `0%` 、 `25%` 、 `50%` 到 `75%` 时，测得 speedup 约为 `1.36x` 、 `1.80x` 、 `1.76x` 、 `1.90x` 。主要趋势是：可复用的 full blocks 越多，fresh prefill 中被跳过的工作越多。 `0%` 也出现 speedup 不应过度解释，它受短 benchmark、block-level cache metadata 和运行时噪声影响；真正重要的是 prefix-hit path 随 shared full blocks 增加而明显降低 prefill 时间。

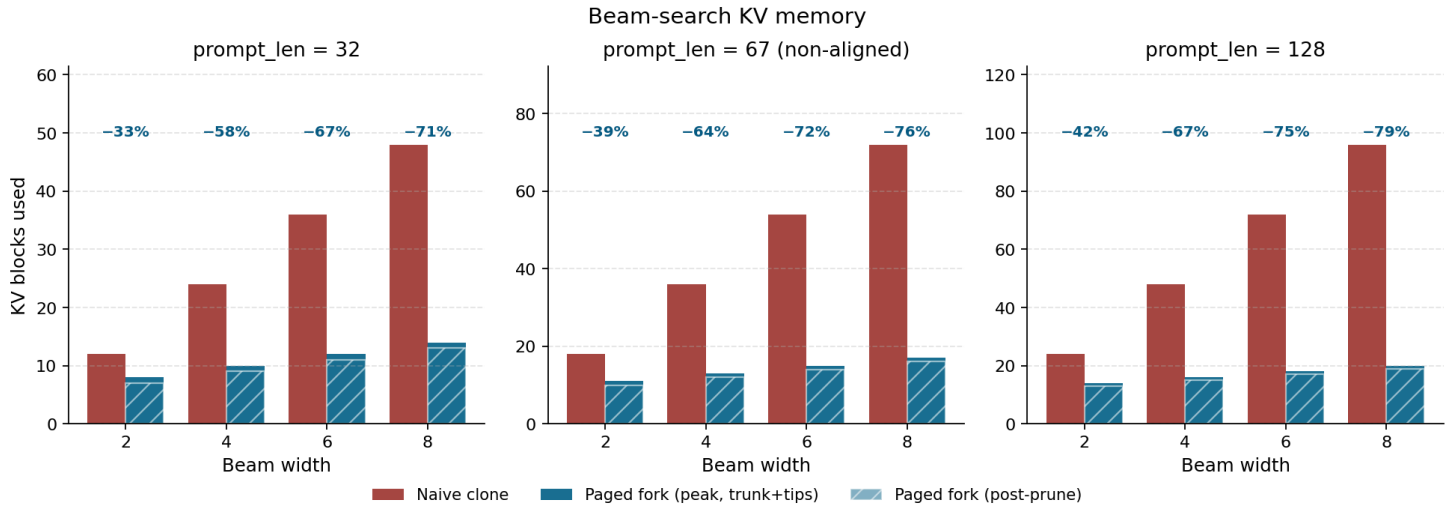
Figure 5：Parallel-sampling KV memory



Parallel sampling 会从同一个 prompt 生成多个 sampled outputs。Naive clone baseline 会为每个 output 复制 prompt KV；paged fork 只共享 prompt blocks，然后为每个 output 的 divergent suffix 分配新 blocks。图中

savings 大约在 `33%--78%` ，prompt 越长、sampled outputs 越多，优势越明显。这个实验说明 block table + reference count 能把“多输出共享 prompt”变成 metadata sharing，而不是 tensor copy。

Figure 6：Beam-search KV memory



Beam-style workload 也有类似结构：多个 beams 共享 prompt，有时还共享已经生成的 trunk。Naive clone 会重复保存每条 beam 的 KV；paged fork 可以共享 prompt/trunk blocks，只为不同分支的 tips 分配 blocks。图中 peak savings 大约 `33%--79%` ，post-prune 后还会进一步降低 live KV blocks。这个实验不是完整 beam-search scheduler，但它清楚展示了 PagedAttention 论文里 branch sharing/copy-on-write 思路的内存收益。

总体结论：Figure 1、2、5、6 主要证明 allocator 和 sharing 的内存收益；Figure 4 证明 prefix reuse 能减少 prefill 工作；Figure 3 则提醒我们，内存管理的优势不自动等于 production-level decode speed，kernel、layout、batching 和 scheduler 仍然决定端到端吞吐。

vLLM/PagedAttention 论文实现 vs 本项目实现

本项目和 vLLM/PagedAttention 论文实现的是同一个核心抽象：把逻辑连续的 KV cache 切成固定大小 blocks，用 block table 把 logical token position 翻译到 physical block/slot，并用 reference count 支持共享。但两者的工程目标和系统边界差别很大。

维度	vLLM / PagedAttention 论文中的 production serving	本项目 MiniTorch 实现
目标	服务真实 LLM 请求， 最大化线上吞吐、降低排队和延迟。	在教学框架中重建核心机制，让 allocator、block table、CUDA lookup 和 benchmark 可读、可验证。
模型和输入	使用真实 pretrained LLM、tokenizer、真实或真实分布的 prompts/outputs。	使用 MiniTorch decoder-only 模型、 随机初始化权重和 synthetic token IDs。
Request scheduler	有 production serving scheduler：continuous batching、admission	benchmark loop 基本是静态 batch，没有完整在线 scheduler、排队模型或 preemption。

维度	vLLM / PagedAttention 论文中的 production serving	本项目 MiniTorch 实现
	control、request lifecycle、memory pressure handling。	
KV cache source of truth	KV cache 是 GPU serving 路径的核心状态，allocator、scheduler 和 kernel 紧密耦合。	完整模型路径里 CPU NumPy `BlockManager` 仍是 authoritative KV cache；CUDA runtime 保存 GPU mirror。GPU-only microbenchmark 则直接测 device-resident runtime。
Memory layout	面向 GPU kernel 优化的 KV block layout，考虑 coalescing、vectorized loads、precision 和长上下文。	清晰优先的 `(num_blocks, block_size, n_head, head_dim)` NumPy/CUDA layout，便于理解但不是最优访存布局。
Attention kernel	production kernel 包含高度优化的 paged attention 路径，支持更复杂的 batching、长 context 和硬件友好实现。	一个可读的 V1-style CUDA decode kernel；没有 vLLM 级别的 template specialization、online softmax、V2 long-context decomposition 或 Nsight 驱动优化。
Host/device transfer	端到端 serving 尽量让模型状态和中间张量留在 GPU，避免每步 host synchronization。	ctypes wrapper 每次 call 仍可能包含 query H2D 和 output D2H；full-model CUDA path 还要从 CPU cache 同步 touched slots/blocks。
Prefix/branch sharing	面向真实 serving workload 的 prefix/block sharing、parallel sampling、beam sharing 和调度策略。	实现 full-block prefix cache、`fork_sequence` / `clone_sequence` 对比和 branch memory simulation；没有完整质量感知 beam-search scheduler。
Precision 和规模	面向 FP16/BF16 等真实推理精度和大模型规模。	主要使用 `float32`，模型规模较小，适合 correctness 和机制验证。
Baseline 和指标	与 production serving baselines 比较 end-to-end request throughput、latency、capacity。	与 no-cache、contiguous KV、stateless full-copy wrapper 比较，指标更偏 allocator efficiency、capacity 和 microbenchmark latency。

因此，本项目可以说“复现了 PagedAttention 的核心内存管理机制”，但不能说“复现了 vLLM 的 production serving environment”。更准确的表述是：本项目验证了 block-granular KV allocation、prefix reuse 和 branch sharing 的机制收益；vLLM 的论文结果还叠加了 production scheduler、真实模型执行、GPU-resident runtime、优化 kernel 和 workload-level batching。

GPU-only KV cache microbenchmark

`project/run_gpu_kv_benchmark.py` 是一个更聚焦的 CUDA runtime microbenchmark。它不走完整 `PagedDecoderLM`，而是直接使用 `PagedAttentionKernel` 的 stateful runtime：先把 K/V cache 上传到 GPU runtime

buffer 一次，然后在计时循环里只调用 decode attention，让 kernel 从 device-resident K/V cache 读取数据。对比项是旧的 stateless wrapper 路径，它每次 attention call 都重新复制完整 host KV cache。

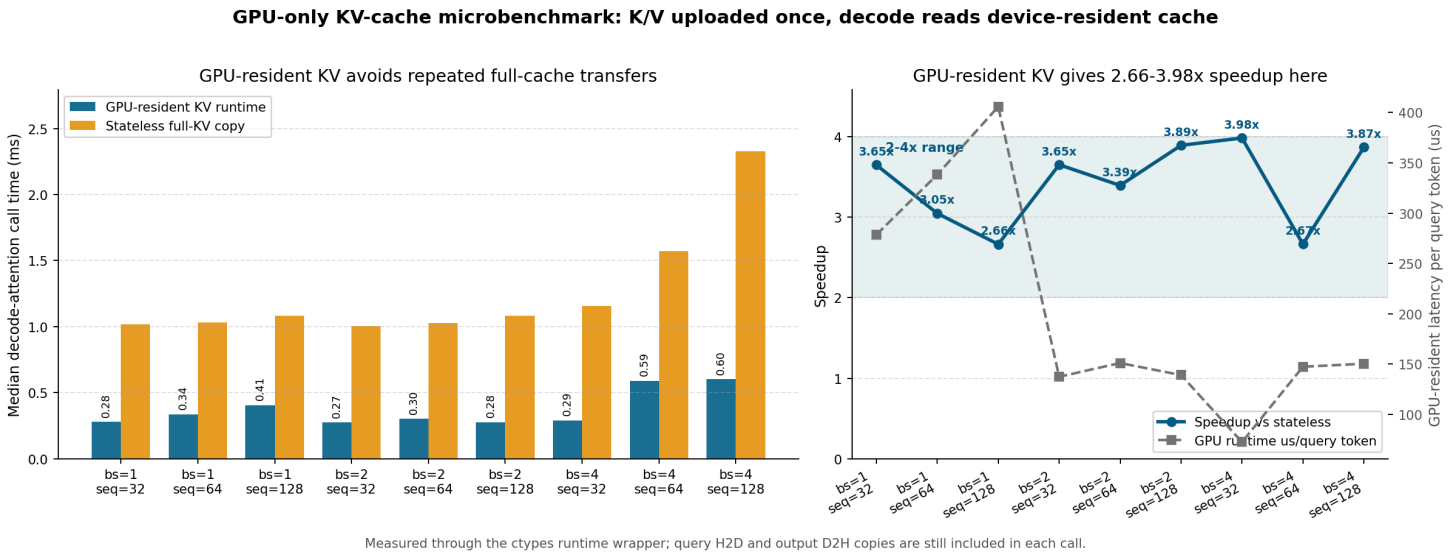
生成 CSV 和图的命令是：

```
python project/run_gpu_kv_benchmark.py \
  --batch-sizes 1 2 4 \
  --seq-lengths 32 64 128 \
  --block-size 16 \
  --n-head 8 \
  --head-dim 64 \
  --warmup-iters 5 \
  --timed-iters 20

python project/plot_gpu_kv_results.py
```

输出文件：

```
benchmarks/results_gpu_kv/gpu_kv_decode_attention.csv
benchmarks/results_gpu_kv/gpu_kv_decode_attention.png
```



图的左半部分显示：当 K/V 已经常驻 GPU runtime buffer 时，decode-attention call 的 median latency 大约是 `0.28--0.60 ms`；而 stateless full-KV-copy 路径大约是 `1.00--2.33 ms`。这说明重复拷贝完整 KV cache 是一个很明显的开销，尤其在 batch size 和 sequence length 变大时更明显。

图的右半部分显示：GPU-resident KV runtime 相比 stateless full-copy wrapper 的 speedup 是 `2.66x--3.98x`，整体落在 PagedAttention/vLLM 论文常见的 `2--4x` headline speedup 区间附近。不过这个比较只说明本项目的 device-resident KV cache 机制能够消除重复 full-cache transfer；它不是对 vLLM production serving throughput 的完整复现。

解读这个图时要注意三个限制：

- 这是 attention runtime microbenchmark，不是完整生成系统 benchmark；它不包含 tokenizer、真实模型权重、scheduler、continuous batching 或真实请求到达过程。
- 本项目的 full-model path 仍然用 CPU `BlockManager` 作为 authoritative KV cache，并把 touched slots/blocks 同步到 CUDA runtime；这个 microbenchmark 刻意绕开完整模型路径，只测 device-resident K/V 读取。
- 当前 ctypes wrapper 每次 call 仍然包含 query host-to-device copy 和 output device-to-host copy，所以图中的 `0.28--0.60 ms` 不是纯 kernel time，而是 runtime wrapper 下的端到端 attention call time。

解读结果时要区分：

- CPU reference decode 主要用于正确性和教学可见性，不代表生产速度。
- CUDA decode 依赖 CUDA 环境和 shared library 编译成功。
- no-cache baseline 和 paged decode 的算法成本模型不同。
- contiguous KV baseline 更适合 decode-speed 公平比较，但仍然过度预留内存。
- prefix-cache 收益取决于 shared prefix length 向下取整到 full blocks。
- 当真实序列长度远小于预留最大长度时，memory savings 最大。

设置和构建

工作目录

所有命令建议从 repo root 执行：

```
cd 11868-course-project
```

脚本会把 `.` 插入 `sys.path`，从别的目录运行可能 import 到错误的 `minitorch`。

Python environment

Linux、WSL 或 bash：

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
```

PowerShell：

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .
```

CPU smoke check

CPU path 使用 MiniTorch `FastOps` 和 Python reference PagedAttention，不需要编译 CUDA kernel：

```
python project/run_inference.py \
  --backend cpu \
  --decode-backend ref \
  --batch-size 1 \
  --prompt-len 8 \
  --max-new-tokens 4
```

CUDA build

Windows 上请在 WSL、Linux shell 或 CUDA-enabled cluster 环境运行，不要直接在普通 PowerShell 里跑 `compile_cuda.sh` 。

如果环境使用 modules：

```
module load cuda/12.4
```

然后：

```
bash compile_cuda.sh
```

CUDA smoke check：

```
python project/run_inference.py \
  --backend cuda \
  --decode-backend cuda \
  --compare-to-ref \
  --batch-size 1 \
  --prompt-len 8 \
  --max-new-tokens 4
```

依赖说明

`requirements.txt` 中重要依赖：

- `numpy` ： KV cache storage 和 numerical reference。
- `pytest` 、 `hypothesis` ： 测试。
- `numba` 、 `pycuda` ： MiniTorch CUDA infrastructure。
- `matplotlib` ： benchmark plots。
- `tqdm` ： 部分 inherited code 的进度工具。

如果 CUDA shared libraries 看起来陈旧：

```
rm -f minitorch/cuda_kernels/*.so
bash compile_cuda.sh
```

运行 inference

`project/run_inference.py` 使用随机 token 和随机初始化权重运行 synthetic generation。它不会加载 tokenizer 或 pretrained weights。

最小 CPU run

```
python project/run_inference.py \
  --backend cpu \
  --decode-backend ref \
  --batch-size 1 \
  --prompt-len 16 \
  --max-new-tokens 8
```

CUDA decode run

编译 CUDA kernels 后：

```
python project/run_inference.py \
  --backend cuda \
  --decode-backend cuda \
  --compare-to-ref \
  --batch-size 2 \
  --prompt-len 32 \
  --max-new-tokens 8
```

Full-recompute correctness check

`--check-correctness` 会把 paged decode logits 和“每步重新 prefill 完整序列”的 reference 比较：

```
python project/run_inference.py \
  --backend cpu \
  --decode-backend ref \
  --check-correctness \
  --batch-size 2 \
  --prompt-len 8 \
  --max-new-tokens 4
```

输出包括：

- `max_abs_error`
- `mean_abs_error`
- `argmax_match_rate`

常用 flags

Flag	默认值	含义
`--n-vocab`	`10000`	synthetic vocabulary size。
`--n-embd`	`256`	embedding dimension，必须能被 `--n-head` 整除。
`--n-head`	`8`	attention head 数。
`--n-layers`	`4`	Transformer layers 数，也是 per-layer KV cache 数。
`--n-positions`	`1024`	position embedding 最大长度。
`--block-size`	`16`	每个 KV block 的 token slots。
`--num-kv-blocks`	`256`	block manager 的物理 block 容量。
`--batch-size`	`1`	同时生成的序列数。
`--prompt-len`	`32`	synthetic prompt length。
`--max-new-tokens`	`64`	生成 token 数。
`--temperature`	`1.0`	sampling temperature； `0` 表示 argmax。
`--backend`	`cpu`	MiniTorch tensor backend： `cpu` 或 `cuda` 。
`--decode-backend`	`ref`	PagedAttention decode backend： `ref` 或 `cuda` 。
`--compare-to-ref`	off	CUDA decode 时同时跑 Python reference 并检查接近。
`--compare-tolerance`	`1e-4`	CUDA/reference 比较 tolerance。
`--check-correctness`	off	与 full-sequence recomputation 比较。

选择 `num-kv-blocks` 时，用：

```
blocks_per_sequence = ceil((prompt_len + max_new_tokens) / block_size)
total_needed ~= batch_size * blocks_per_sequence
```

如果 `num_kv_blocks` 太小，block manager 会报 `No free blocks available` 。

测试和 Benchmark

Unit tests

运行所有 tests：

```
pytest tests/ -q
```

聚焦运行：

```
pytest tests/test_block_manager.py -q
pytest tests/test_paged_attention.py -q
pytest tests/test_parity.py -q
pytest tests/test_benchmark.py -q
```

CUDA-specific tests 在 CUDA 或 compiled shared library 不可用时会自动 skip。

Quick benchmark smoke run

先用很小配置：

```
python project/run_benchmark.py \
  --batch-sizes 1 \
  --seq-lengths 16 \
  --block-sizes 8 \
  --max-new-tokens 4 \
  --skip-max-batch
```

默认输出到 `benchmarks/results/` 。

General benchmark

```
python project/run_benchmark.py \
  --batch-sizes 1 2 4 \
  --seq-lengths 32 64 128 \
  --block-sizes 8 16 \
  --max-new-tokens 16 \
  --compare-baseline \
  --compare-prefix-cache
```

主要输出：

```
benchmarks/results/benchmark_results.csv
benchmarks/results/fragmentation_results.csv
```

绘图：

```
python project/plot.py
```

Rigorous benchmark

```
python project/run_rigorous_benchmark.py \
  --output-dir benchmarks/results_rigorous \
  --backend auto \
  --decode-backend auto
```

更快的开发检查：

```
python project/run_rigorous_benchmark.py \
  --output-dir benchmarks/results_rigorous_smoke \
  --backend cpu \
  --decode-backend ref \
  --skip-decode \
  --skip-prefix
```

绘制 rigorous figures：

```
python project/plot_rigorous_figures.py
```

GPU-only KV cache microbenchmark 和绘图：

```
python project/run_gpu_kv_benchmark.py \
  --batch-sizes 1 2 4 \
  --seq-lengths 32 64 128 \
  --block-size 16 \
  --n-head 8 \
  --head-dim 64 \
  --warmup-iters 5 \
  --timed-iters 20

python project/plot_gpu_kv_results.py
```

常见 metrics

Metric	含义
`tokens_per_sec`	end-to-end generated tokens per second，包含 prefill 和 decode。
`decode_tokens_per_sec`	decode-only generated tokens per second。
`time_per_token_ms`	平均 decode latency。
`decode_p50_ms` ， `decode_p95_ms`	median 和 p95 decode latency。
`internal_frag`	tail blocks 内浪费 slot 的比例。
`kv_efficiency`	live KV bytes / reserved paged KV bytes。
`memory_savings_ratio`	相比 naive contiguous reservation 的节省比例。
`correctness_pass`	output 是否在 tolerance 内匹配 reference。
`prefix_prefill_speedup`	fresh prefill time / cached-prefix prefill time。

Benchmark 建议：

- 先小配置跑通，再扩大模型和序列长度。
- 改 CUDA decode 时使用 `--compare-to-ref` 。
- 只有验证过配置后再用 `--skip-correctness` 。
- throughput-only 迭代时可用 `--skip-max-batch` 。
- 保证 `n_embd` 能被 `n_head` 整除。
- 确保 `num_kv_blocks` 足够容纳 `batch_size * ceil((seq_len + max_new_tokens) / block_size)` 。

API 参考

`minitorch.block_manager`

`KVBlock(block_id, block_size, n_head, head_dim)` ：一个 physical KV block 的 metadata。

重要属性：

- `block_id`
- `block_size`
- `ref_count`
- `num_filled`
- `n_head`
- `head_dim`
- `block_hash`
- `is_prefix_cached`

`BlockTable(seq_id, block_ids=None)`：每个序列的 logical-to-physical block map。

`PrefixCacheMatch`：prefix lookup 返回的数据类，字段是 `block_ids` 和 `cached_token_count`。

`BlockManager(...)` 构造：

```
BlockManager(
    num_blocks,
    block_size,
    n_head,
    head_dim,
    num_layers=1,
    cache_dtype=np.float32,
)
```

核心 allocation：

- `allocate_block() -> KVBlock`
- `free_block(block_id) -> None`
- `allocate_blocks_for_sequence(seq_id, num_tokens) -> BlockTable`
- `append_token_to_sequence(seq_id) -> KVBlock`
- `free_sequence(seq_id) -> None`

K/V access：

- `write_kv_slot(layer_id, block_id, slot_idx, key, value) -> None`
- `write_token_kv(seq_id, layer_id, key, value) -> tuple[int, int]`
- `get_physical_location(seq_id, token_index) -> tuple[int, int]`
- `get_block_table_array(seq_ids, pad_value=-1) -> np.ndarray`
- `get_context_len(seq_id) -> int`

Metrics：

- `get_num_free_blocks() -> int`
- `get_num_used_blocks() -> int`
- `compute_fragmentation() -> dict`
- `compute_kv_memory(max_seq_len) -> dict`

Prefix cache：

- `compute_block_hash_chain(token_ids, extra_hash=None) -> list[str]`
- `lookup_prefix_blocks(token_ids, extra_hash=None) -> PrefixCacheMatch`
- `allocate_sequence_with_prefix(seq_id, num_tokens, prefix_match) -> BlockTable`
- `publish_sequence_prefix_blocks(seq_id, token_ids, extra_hash=None) -> None`
- `evict_cached_block_if_needed() -> bool`

minitorch.paged_attention

`standard_attention(query, key, value, mask=None)`：连续 scaled dot-product attention。返回 `(batch, n_head, seq_q, head_dim)`。

`paged_attention_ref(...)`：reference PagedAttention，通过 block tables gather K/V，再调用 `standard_attention`。

`PagedAttentionKernel`： `minitorch/cuda_kernels/paged_attention.so` 的 ctypes wrapper。重要方法：

- `ensure_runtime(...)`
- `upload_layer_cache(key_cache, value_cache)`
- `update_slot(block_id, slot_idx, key, value)`
- `upload_block(block_id, key_block, value_block)`
- `update_metadata(block_tables, context_lens)`
- `forward(...)`
- `close()`

`PagedMultiHeadAttention(...)`：Transformer 使用的 attention module。重要方法：

- `forward_prefill(x, block_manager, seq_ids)`
- `forward_prefill_with_prefix_batch(...)`
- `forward_decode(x, block_manager, seq_ids)`
- `close_decode_runtime()`

minitorch.transformer

`FeedForward(n_embd, p_dropout=0.0, backend=None)`：Transformer MLP block。

`PagedTransformerLayer(...)`：一个 transformer layer，提供：

- `forward_prefill(x, block_manager, seq_ids)`
- `forward_decode(x, block_manager, seq_ids)`

`PagedDecoderLM(...)`：完整 decoder-only LM，提供：

- `forward_prefill(idx, block_manager, seq_ids)`
- `forward_decode(idx, block_manager, seq_ids, start_pos=0)`
- `close_decode_runtime()`
- `generate(model, idx, max_new_tokens, block_manager, temperature=1.0)`

术语表

MiniTorch 和 Tensor

术语	含义
Tensor	MiniTorch 的多维数组对象，用于模型计算。
TensorData	Tensor 背后的 storage、shape、stride 表示。
Shape	维度大小，例如 `(batch, seq_len, n_embd)` 。
Stride	某个维度索引变化时，在 storage 中跳过的位置数。
Contiguous	Tensor 元素按预期 row-major 顺序排列，适合 `view` 。
Backend	MiniTorch 执行实现，例如 `FastOps` 或 `CudaKernelOps` 。
Module	MiniTorch 中拥有参数和 train/eval 状态的 layer 类。
Parameter	`Module` 拥有的可训练值。
`tensor_from_numpy`	把 NumPy array 包成 MiniTorch Tensor。
`to_numpy`	把 MiniTorch Tensor 转成 NumPy array。

Transformer

术语	含义
Decoder-only model	只使用过去 token 作为上下文预测 next token 的语言模型。
Token embedding	token ID 对应的学习向量。
Positional embedding	token position 对应的学习向量。
Hidden state	Transformer 层之间传递的 per-token vector。
Query (`Q`)	当前 token 提出“我要找什么”的向量。
Key (`K`)	每个 token 用于匹配的向量。
Value (`V`)	根据 attention weights 被复制的信息向量。
Attention score	query 和 key 的点积，除以 `sqrt(head_dim)` 。
Attention weight	score 经过 softmax 后的概率权重。
Multi-head attention	把 attention 分成多个 head，每个 head 使用较小的 `head_dim` 。
Causal mask	阻止 token attend 到未来位置的 mask。

术语	含义
Logits	vocabulary 上未归一化的 token scores。

Inference 和 KV cache

术语	含义
Prefill	初始阶段，处理完整 prompt 并填充 KV cache。
Decode	自回归阶段，每次处理一个新 token。
KV cache	保存历史 token 的 key/value vectors。
Context length	某个序列当前拥有的 token 数。
Contiguous KV cache	每个序列预留一整段连续 cache。
Paged KV cache	用固定大小物理 blocks 存储序列 KV。
Internal fragmentation	allocated blocks 内未使用的 tail slots。
External fragmentation	used block 范围之间的 free blocks；本项目只是简单诊断。

Block manager

术语	含义
Physical block	`BlockManager.key_cache` 和 `value_cache` 中真实的存储 block。
Logical block	某个序列内部由 token position 计算出的 block index。
Block table	每个序列的 logical block 到 physical block id 映射。
`seq_id`	active sequence/request 的 identifier。
`block_id`	全局物理 block pool 中的 identifier。
Slot	一个 physical block 内的 token 位置。
`num_filled`	physical block 中当前有效 token slots 数。
`ref_count`	引用该 block 的 active/cached owners 数，用于共享安全。
Free list	普通未分配 physical block id 列表。
Prefix cache	full-block token hash 到 reusable physical block 的映射。
Full block	包含 `block_size` 个 token 的完整 block；只有 full block 会发布到 prefix cache。
Hash chain	每个 block hash 依赖上一个 block hash 和当前 block tokens 的前缀缓存 hash。

术语	含义
LRU eviction	least-recently-used 回收；本项目用于 cached-free blocks。

PagedAttention 和 CUDA

术语	含义
Standard attention	在连续 K/V Tensor 上做 attention。用于 prefill 和 reference。
Paged attention	通过 block table 读取分散 K/V 的 attention。
Gather	把分散 K/V blocks 按逻辑 token 顺序读取出来。
Reference backend	Python/NumPy/MiniTorch 参考实现，flag 中是 `--decode-backend ref`。
CUDA backend	自定义编译 PagedAttention kernel，flag 中是 `--decode-backend cuda`。
Stateless kernel call	每次都传 host cache 和 metadata 的 CUDA call。
Stateful runtime	设备端持有 cache buffers，只更新变化 blocks/metadata 的 runtime。
Kernel	GPU 上运行的函数。
Grid	一次 kernel launch 的 CUDA thread blocks 集合。
Thread block	一组可同步并共享 shared memory 的 CUDA threads。
Warp	NVIDIA GPU 中 32 个一起执行的 threads。
Shared memory	一个 thread block 内可见的快速片上内存。
Warp reduction	在一个 warp 内做 sum/max 等归约。

常见故障和排查

问题	常见原因和处理
`Sequence X is already active`	同一个 `seq_id` 已经 prefill 过但还没 `free_sequence`。完成 generation 后释放序列。
`forward_decode expects a single new token`	decode 输入的 `seq_len` 不是 1。
`No free blocks available`	`num_kv_blocks` 太小，或者 prefix cached blocks 无法回收。增大 block 数或缩小 batch/length。
CUDA 找不到设备	检查 CUDA 可见性、import order、`--cudart shared`、是否加载本仓库 `minitorch`。

问题	常见原因和处理
`paged_attention.so` 找不到	运行 `bash compile_cuda.sh` ，并确认输出在 `minitorch/cuda_kernels/` 。
benchmark baseline 报错	某些 baseline 只支持特定 layer 数； 先读错误信息和脚本参数。
手写 decode loop 的位置错	`start_pos` 应该等于当前 decode token 的绝对位置。

修改代码前该读哪里

目标	先读
block allocation、prefix cache、memory metrics	本文件的 “BlockManager 详解” 和 ../minitorch/block_manager.py。
Python reference attention 或 MHA wrapper	本文件的 “Python PagedAttention 详解” 和 ../minitorch/paged_attention.py。
CUDA kernel 或 ctypes runtime	本文件的 “CUDA Kernel 和 Runtime” 和 ../src/paged_attention.cu。
model prefill/decode/generate	本文件的 “Transformer 集成” 和 ../minitorch/transformer.py。
CLI flags、tests、benchmark outputs	本文件的 “运行 inference” 和 “测试和 Benchmark”。

最后复盘

读完整个项目时，可以把调用链压缩成两条：

Prefill：

```
PagedDecoderLM.forward_prefill
  -> BlockManager.lookup_prefix_blocks
  -> allocate_blocks_for_sequence or allocate_sequence_with_prefix
  -> _embed
  -> PagedTransformerLayer.forward_prefill
       -> PagedMultiHeadAttention.forward_prefill
            -> standard_attention
            -> write K/V into BlockManager
  -> publish_sequence_prefix_blocks
  -> final ln + lm_head
```

Decode：

```
PagedDecoderLM.forward_decode
  -> _embed(start_pos)
  -> BlockManager.append_token_to_sequence
  -> PagedTransformerLayer.forward_decode
       -> PagedMultiHeadAttention.forward_decode
            -> write new K/V slot
            -> paged_attention_ref or CUDA runtime forward
  -> final ln + lm_head
```

只要记住 block table 的翻译规则，就能把大多数代码串起来：

```
logical_block = token_index // block_size
slot_idx      = token_index % block_size
block_id      = block_table[logical_block]
```

PagedAttention 的漂亮之处就在这里：模型仍然按连续序列理解上下文，而内存系统可以像分页一样灵活地分配、复用和回收 KV blocks。